



Distributed Systems

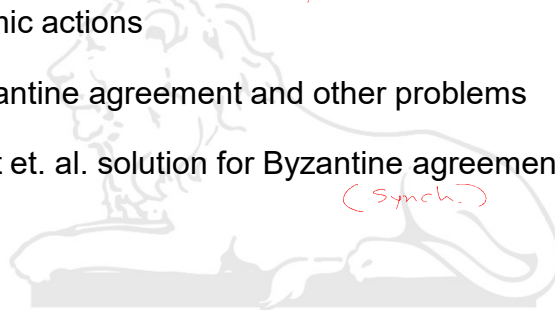
Fault Tolerance & Recovery



Recap



- Some building blocks
 - Agreement *type of fault*
 - synch. / Asynch.*
 - signed / oral*
 - Atomic actions
- The Byzantine agreement and other problems
- Lamport et. al. solution for Byzantine agreement *(synch.)*



Byzantine Agreement Problem

- total n processes, at most m of which can be faulty
- no (deterministic) solution is possible if
 - asynchronous system, or
 - $n < (3m + 1)$ where **n – total processors and m - faulty** (synchronous system)
- needs at least $(m+1)$ rounds of message exchange (lower bound result)
- “Oral” messages – messages can be forged / changed in any manner, but the receiver always knows the sender
- Assumption: agreement is to be reached between 0 and 1

Byzantine generals algorithm



Boolean: $v \leftarrow$ initial value;

integer: $f \leftarrow$ max no. of malicious processes, $\leq \lfloor (n-1)/3 \rfloor$;

(message type) - OM(v , Dests, List, faulty), where

v - is a Boolean,

Dests - set of dest. process ids to which the message is sent,

List - list of process ids traversed by this message, ordered from most recent to earliest,

faulty - an integer indicating the number of malicious processes to be tolerated.

Oral_Msg (f), where $f > 0$:

- (1) Commander initiates and sends his source value v to all other processes using a OM(v , N , $\langle i \rangle$, f) message.

The commander returns his own value v and terminates.

Byzantine generals algorithm



(2) [Recursion unfolding]:

For each message of the form $OM(v_j, Dests, List, f')$ received in this round from some process j , the process i uses the value v_j it receives from the source j , and using that value, acts as a new source. (If no value is recd, a default value is assumed.)

To act as a new source, the process i initiates Oral_Msg ($f' - 1$), wherein it sends

$OM(v_j, Dests - \{i\}, \text{concat}(\langle i \rangle, L), (f' - 1))$

to destinations not in $\text{concat}(\langle i \rangle, L)$ in the next round.

Byzantine generals algorithm



(3) [Recursion folding]:

For each message of the form $OM(v_j, Dests, List, f')$ received in step 2, each process i has computed the agreement value v_k , for each k not in $List$ and $k \neq i$, corresponding to the value received from P_k after traversing the nodes in $List$, at one level lower in the recursion. If it receives no value in this round, it uses a default value.

Process i then uses the value majority _{$k \in List, k \neq i$} (v_j, v_k) as the agreement value and returns it to the next higher level in the recursive invocation.

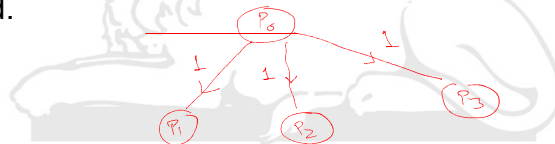
Byzantine generals algorithm



Oral_Msg(0):

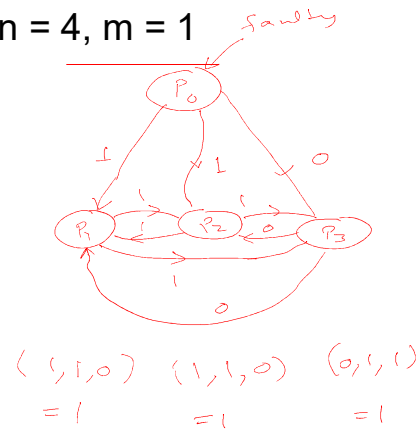
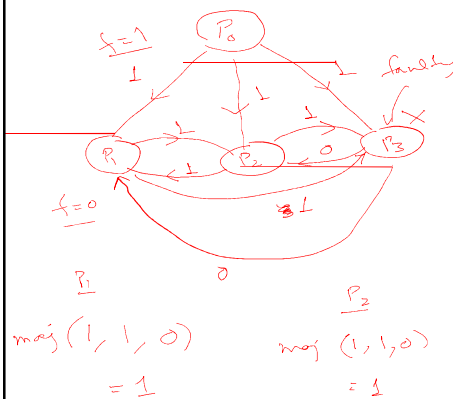
(1) **[Recursion unfolding:]** Process acts as a source and sends its value to each other process.

(2) **[Recursion folding:]** Each process uses the value it receives from the other sources, and uses that value as the agreement value. If no value is received, a default value is assumed.



IIT ROORKEE

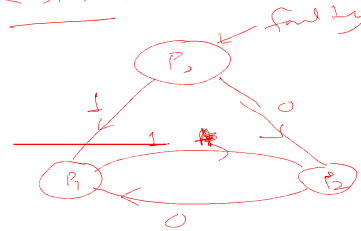
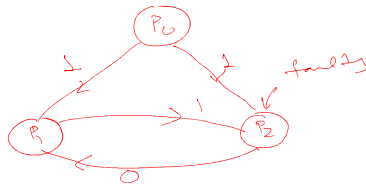
Byzantine agreement: $n = 4, m = 1$



Byzantine agreement can't be reached if $n < 3m + 1$

Example: $n = 3, m = 1$

$$3 < 3 \times 1 + 1$$



- Time complexity = $m+1$ rounds
- Message Complexity = $O(n^m)$
- One can reduce message complexity to polynomial by increasing time

An iterative Algorithm



(variables)

boolean: $v \leftarrow$ initial value;

integer: $f \leftarrow$ maximum number of malicious processes, $\leq \lfloor (n-1)/3 \rfloor$;

tree of boolean:

- level 0 root is v_{init}^L , where $L = \langle \rangle$;
- level h ($f \geq h > 0$) nodes: for each v_j^L at level $h-1 = \text{sizeof}(L)$, its $n-2-\text{sizeof}(L)$ descendants at level h are $v_k^{\text{concat}(\langle j \rangle, L)}$, $\forall k$ such that $k \neq j$, i and k is not a member of list L .

(message type)

$OM(v, Dests, List, faulty)$, where the parameters are as in the recursive formulation.

(1) Initiator (i.e., commander) initiates the oral Byzantine agreement:

(1a) send $OM(v, N - \{i\}, \langle P_i \rangle, f)$ to $N - \{i\}$;

(1b) return(v).

(2) (Non-initiator, i.e., Lieutenant) receives Oral Message OM :

(2a) for $rnd = 0$ to f do

(2b) for each message OM that arrives in this round, do

(2c) receive $OM(v, Dests, L = \langle P_{k_1} \dots P_{k_{f+1-faulty}} \rangle, faulty)$ from P_{k_1} ;
 $// \text{ faulty} + \text{round} = f, |Dests| + \text{sizeof}(L) = n$

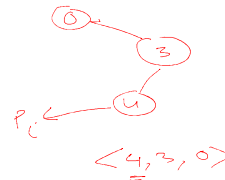
(2d) $v_{\text{head}(L)}^{\text{tail}(L)} \leftarrow v$; $// \text{ sizeof}(L) + \text{faulty} = f + 1$. fill in estimate.

(2e) send $OM(v, Dests - \{i\}, \langle P_i, P_{k_1} \dots P_{k_{f+1-faulty}} \rangle, faulty - 1)$ to $Dests - \{i\}$ if $rnd < f$;

(2f) for $level = f - 1$ down to 0 do

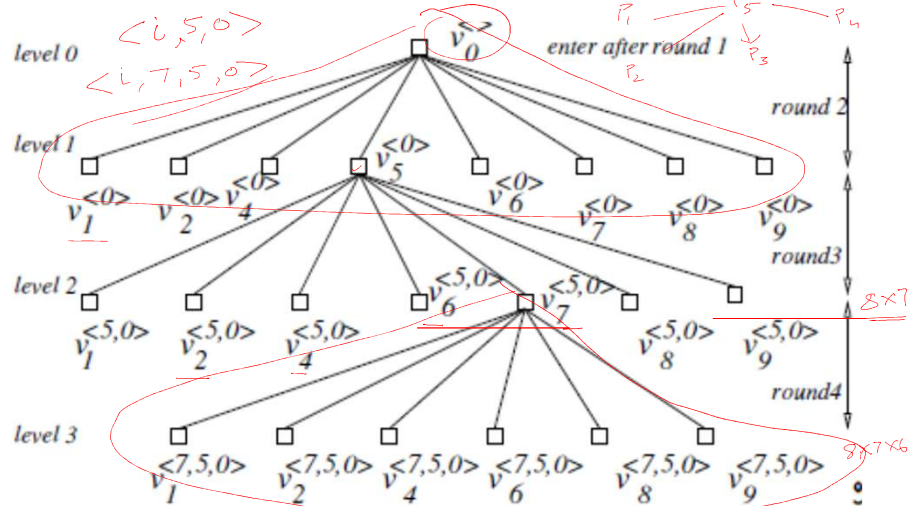
(2g) for each of the $1 \cdot (n-2) \cdot \dots \cdot (n - (level + 1))$ nodes v_x^L in level $level$, do

(2h) $v_x^L(x \neq i, x \notin L) = \text{majority}_y \notin \text{concat}(\langle x \rangle, L); y \neq i; (v_x^L, v_y^{\text{concat}(\langle x \rangle, L)})$;



Working at process P_3

- Process P_0 is the source
- $n = 10$ and $m = 3$



Error Recovery

- Forward-error Recovery not exact fault & how it can be corrected
- Backward-error Recovery
 - Adv:** simpler, independent of the fault
 - Disadv:** Performance penalty, fault may occur again, some components of system state may be unrecoverable

Backward-error Recovery



- Operation based approach

All the modifications made to the state of the process are recorded in sufficient detail (audit trail or log)

- State-based approach

Complete state of the process is saved when a recovery point is established (checkpoint)

- Assume a secondary storage and a stable storage

IIT ROORKEE

15

System Model

